RESTORING A VISIBLE MESS

Four Independent Roblox Prototype Specifications — v1.0

Prepared: 2026-11-06
Status: Proposed design for owner review. Not implemented or playtested.
Owner decision required: Choose which prototype to authorize, or authorize an ordered set of experiments.
Relationship to Thaw a Creature: None. These are separate research prototypes, not changes to its canonical design, code, progression, or development plan.

What this pack contains

Code	Standalone specification	Core player fantasy	Assumed P0 effort
A	`01_Environmental_Clearing_Prototype_v1.0.md`	I can make an overwhelming mess disappear	2–3 developer-days
B	`02_Object_Organization_Prototype_v1.0.md`	I can turn a chaotic space into something orderly	3–5 developer-days
C	`03_Restoration_Reveal_Prototype_v1.0.md`	I can uncover something surprising by restoring it	3–5 developer-days
D	`04_Precision_Washing_Prototype_v1.0.md`	I can precisely wash grime from a dirty object	6–10+ developer-days; stop/go technical spike
Shared	`05_Shared_Research_and_Playtest_Protocol_v1.0.md`	Apples-to-apples validation	Shared method, not a fifth game

A developer-day is an estimate of six focused engineering hours, not an observed competitor build time. Budgets exclude public-launch polish, marketing, monetization, production art, retention economies, and large-scale QA. No development time is guaranteed.

Scope discipline

Each prototype is **one task and a replay button**, with a target initial playthrough of **45–90 seconds** (tuning hypothesis). One tester per instance is sufficient for P0; server authority still applies. There are **no currency systems, upgrades, persistence, inventory collections, monetization, AI-generated content pipelines, cross-game integration, trading, or live events**.

All numeric values in these documents are labeled **first-pass experiment assumptions**. Changing them after real playtesting is expected; doing so should not turn a prototype into a production game without an explicit decision.

Read order

1. Read `05_Shared_Research_and_Playtest_Protocol_v1.0.md` to understand success metrics and comparison fairness.

2. Read the individual specs A–D, each with user experience, systems, authority, work plan, fallback, QA, and stop/go gate.
3. Choose one to implement. Prototype A is the lowest engineering-risk first test; C tests anticipation; B tests sorting; D is a high-risk fidelity benchmark.
4. Record observed results in the shared scorecard *before* planning expansions.

Authority and status

- Each spec is a **proposal**, not owner-approved scope or a claim that Roblox Studio work exists.
- For gameplay and acceptance, the relevant individual spec governs that prototype. The shared protocol governs comparison and telemetry definitions.
- Where estimates, implementation approaches, or numerical targets conflict, obtain owner approval and record the revision; **do not let an AI coding assistant silently resolve the conflict**.
- Do not import designs, assets, cash systems, creatures, or code from *Thaw a Creature* without a separate authorized request.

Technical sources

Official API documentation (checked 2026-11-06):

- Roblox raycasting: https://create.roblox.com/docs/workspace/raycasting
- ProximityPrompt: https://create.roblox.com/docs/reference/engine/classes/ProximityPrompt
- CollectionService: https://create.roblox.com/docs/reference/engine/classes/CollectionService
- Cross-platform input: https://create.roblox.com/docs/reference/engine/classes/ContextActionService
- EditableImage technical constraints (optional, prototype D only): https://create.roblox.com/docs/reference/engine/classes/EditableImage

File formats

Every specification and the shared testing protocol is provided in editable Markdown and a readable PDF. The `.zip` archive includes all files. Markdown is the preferred implementation handoff format to Claude Code / Codex; PDFs are for human review.

RESTORATION PROTOTYPES — SHARED RESEARCH & PLAYTEST PROTOCOL

v1.0 | Four comparable Roblox experiments

Status: Proposed; not implemented, measured, or approved.
Date: 2026-11-06
Applicability: Prototypes A, B, C, D only. This document does not authorize new features.

1. Research objective and falsifiable hypothesis

Question: Which of four restoration interactions delivers the strongest observable satisfaction and voluntary replay relative to actual engineering and content-authoring cost?

Hypothesis: At least one interaction will be understandable without instruction, enable most new testers to complete a short task, and motivate a substantial share to replay **without currencies or upgrades**.

Disconfirmation: Testers repeatedly need explanation, do not complete, report the action is tedious, or only express interest after requesting external rewards. A visually attractive final screenshot is not sufficient validation.

2. Control the experiment

Variable	Required policy
Platform	Roblox Studio test place; each prototype isolated from Thaw a Creature
Audience	First-time testers unfamiliar with the particular prototype
Sample	Suggested 10–15 participants per prototype for qualitative issue discovery; **not** statistical proof
First round	Target 45–90 sec; tune content density; measure actual time
Loop	Observe mess → interact → visible change → 100% transformation → replay choice
Controls	Desktop and touch screen usable; document device for every test
Team	Same developer skill assumptions and time-logging conventions for each
Progression	No currency, no persistent inventory, no upgrades, no automation
Social	Single player P0; no networking benefits or social rewards in one prototype only
Visual fidelity	Similar low-poly/blockout clarity and equivalent basic sound feedback
Guidance	One short in-world instruction, no forced tutorial cutscene
Second round	Same action, reseeded or changed content; no new mechanic introduced

Test order can bias results. If using the same participants, rotate which prototype they try first. Do not present A then C to everyone. An additional within-prototype comparison for C (hidden vs disclosed result) is optional and **must be logged as a separate variant**, because C intentionally introduces anticipation not present in A.

3. Common state & server authority contract

All prototypes use the logical states:

`READY → ACTIVE → COMPLETE → REPLAY_OFFERED → RESETTING → ACTIVE`

- Each round has an immutable, server-generated `RoundId`, `StartedAt`, configuration version, and completion condition.
- The **server alone** accepts cleaning, placement, or reveal progress and decides completion. Clients may predict cosmetic feedback but must reconcile to authoritative results.
- The server validates action rate, legitimate target ID, distance, round ID, active state, and the target's not-already-completed state. Bad or repeated requests cannot increment progress twice.
- Reset creates a new round ID and resets all relevant objects, not merely the HUD percentage. Requests from old rounds are rejected.
- Do not use DataStore for P0. Test-session counters are in memory and can be printed as structured logs at the end.
- When a player leaves or their character resets mid-task, clean up held objects, active actions, and session state. The prototype should be recoverable with a fresh round.

Suggested small architecture (not mandatory exact filenames)

```
ReplicatedStorage/
  RestorationShared/Config.lua
  RestorationShared/Remotes/ActionRequested
  RestorationShared/Remotes/StateChanged
ServerScriptService/
  RestorationServer.server.lua        -- bootstrap only
  RestorationModules/RoundService.lua
  RestorationModules/ActionValidator.lua
  RestorationModules/MetricLogger.lua
StarterPlayer/StarterPlayerScripts/
  RestorationClient.client.lua        -- bootstrap only
  RestorationClientModules/InputController.lua
  RestorationClientModules/FeedbackController.lua
  RestorationClientModules/HUDController.lua
Workspace/
  PrototypeArena/
```

Prototype-specific modules should replace/extend the action module instead of creating redundant script clones attached to every prop. This shared architecture is a guide, not a dependency on an elaborate framework.

4. Shared telemetry event definitions

Event	Required fields	When emitted
`RoundStarted`	SessionId, Prototype, Variant, RoundId, Device, ConfigVersion, timestamp	Server initializes playable round
`FirstValidAction`	RoundId, elapsedSeconds	First server-accepted meaningful interaction
`ProgressChanged`	RoundId, progress01, elapsedSeconds	Server accepts progress; sample or coalesce; avoid console spam
`RoundCompleted`	RoundId, elapsedSeconds, acceptedActions	Server marks 100% complete
`ReplayOffered`	RoundId, elapsedSeconds	Clear repeat choice is displayed
`ReplayChosen`	previous RoundId, new RoundId	Player voluntarily starts second round
`SessionEnded`	RoundId, result, elapsedSeconds	Player leaves, tester stops, or recording ends
`RequestRejected`	reason, actionType	Aggregate counts, not high-frequency per event logs

Use a session-scoped random test ID, not personal identifiers in shared reports. No player-age inference, purchase data, or secret identifiers. Record tester feedback on a separate consented sheet; do not add network tracking without approval.

5. Common performance and accessibility checks

- Desktop: mouse + keyboard; mobile: touch controls in a real or emulated mobile viewport. Gamepad is a desirable check if time allows, not P0 in this pack.
- Action targets must be visually legible. Do not rely exclusively on red vs green. Use text/progress numbers and shape/visibility change.
- Avoid flashing effects; allow basic VFX/sound toggle if any test participant needs it.
- Art budget first pass: aim for an understandable blockout with stable performance on a mid-range phone. Collect FPS and frame time; do not claim a platform-wide performance guarantee from one emulator.
- No client should be able to increase total progress by spamming remotes, forging completed status, or acting beyond reach.

6. Measurement formulas and interpretation

Metric	Exact definition	Proposed first-pass interpretation
Time to first action	`FirstValidAction - RoundStarted`	Desirable median <=10 sec
Completion rate	Unique first rounds completed / eligible first rounds started	Exploratory target >=75%
Completion duration	`RoundCompleted - RoundStarted` on completed rounds	Aim 45–90 sec, but do not optimize solely for duration
Voluntary replay rate	Players who choose an offered second round / players shown a replay offer	Exploratory target >=50%
Self-reported satisfaction	Standard question, 1–5: “How satisfying was the transformation itself?”	Explore average >=4.0

Metric	Exact definition	Proposed first-pass interpretation
Comprehension failure	Tester needs explanation or uses wrong action >2 times	Flag repeated patterns, not just mean
Content authoring effort	Minutes to construct and QA the second playable variation	Lower is better at comparable quality
Actual implementation time	Focused work hours P0 + bugs + mobile pass	Count debugging and failed experiments

Warning: These cutoffs are *hypotheses chosen for decision-making*, not Roblox market benchmarks. A 10–15-person convenience sample cannot establish true retention, product-market fit, revenue, or significance. Include raw numbers (e.g., 8/12), not percentages alone. Ask players to show behavior rather than relying only on stated intent.

7. Common scripted observation protocol

1. Ask the tester to start without prior game explanation beyond the on-screen label.
2. Record the first meaningful action and whether it occurred without prompting.
3. Observe areas of confusion, repeated actions, device issues, navigation problems, and unintended successes.
4. Record the completion result and elapsed time.
5. Offer a neutral **"Try another round"** button beside a neutral **"Finish"** button. Do not advertise prizes or create false urgency.
6. Observe whether a second round begins voluntarily; do not push.
7. Ask the same two questions: "How satisfying was the transformation itself, from 1 to 5?" and "What part felt best or worst?"
8. Record the real time needed to author a second content variation of comparable quality.

8. Cross-prototype decision scorecard

Complete only after the same kind of playtest exists for each. A suggested rubric, **not a prediction**, is:

Criterion	Weight	Scoring principle
Voluntary replay	30%	Higher observed replay counts, accounting for sample size
First-round comprehension/completion	25%	Can new players act, understand progress, and finish?
Satisfying transformation	20%	Directly observed reactions + common 1–5 question
Focused development cost	15%	Actual implementation hours and technical risks
Cost of content expansion	10%	Real second-variant authoring time and QA

Keep qualitative notes beside numbers. A one-point difference on a synthetic weighted score is not evidence that one prototype will earn more on Roblox. Any candidate that fails the **core interaction** should not be rescued by hypothetical monetization systems.

9. Decision gates and stop rules

Gate G0: Input and visual clarity. First-time tester can perform action without a verbal explanation. If not, fix cues before adding progression.

Gate G1: Functional correctness. No double-counted progress, no stale-round events, no stuck held objects, no false 100%, no forced crashes.

Gate G2: Emotional payoff. Players can clearly point to what became better and at least some voluntarily complete a second round.

Gate G3: Content scalability. A second variant must be built with measured effort. If authoring 10 variants requires 10 different systems, the product is not low-cost despite the P0 prototype.

Gate G4: Selection. Compare actual data, choose one **if evidence differentiates**, or run a targeted second experiment if results are ambiguous. Do not plan upgrades, currencies, or retention systems before this decision.

10. Sources and scope

Roblox Creator Hub (technical support, verified 2026-11-06):

- https://create.roblox.com/docs/workspace/raycasting
- https://create.roblox.com/docs/reference/engine/classes/ProximityPrompt
- https://create.roblox.com/docs/reference/engine/classes/CollectionService
- https://create.roblox.com/docs/reference/engine/classes/ContextActionService
- https://create.roblox.com/docs/reference/engine/classes/EditableImage

Market motivation was identified in the separately prepared `Roblox_Market_Intelligence_Research_Report_2026-11-06.md`. This shared protocol does not claim that reference games launched with these minimal features, or that their development costs are publicly known.

PROTOTYPE A — ENVIRONMENTAL CLEARING

Roblox Minimum Functional Fantasy Specification | v1.0

Status: PROPOSED FOR OWNER REVIEW; NOT IMPLEMENTED / NOT PLAYTESTED
Date: 2026-11-06
Prototype code: REST-A
Primary fantasy: "I can turn a messy space into a visibly clean space through my own actions."
Reference products (inspiration only): *Clean the Moon!*, *Clean all the Leaves*, *Mow The Lawn!*
First-pass effort assumption: 2–3 focused developer-days (6 hours/day), plus participant recruitment and real-world playtesting.
Shared authority: `05_Shared_Research_and_Playtest_Protocol_v1.0.md`. No relationship to *Thaw a Creature*.

1. Decision this prototype must answer

Can removing simple, repeated debris patches be intrinsically satisfying enough that a new Roblox player completes one cleanup and voluntarily starts another, without progression or currency?

- **Hypothesis A-H1:** Spatially localized removal plus strong before/after readability motivates completion without an external reward.
- **Hypothesis A-H2:** A new debris layout reuses the system cheaply enough for many content variations.
- **Failure criterion:** Most participants treat cleaning as repetitive labor, require an external reward, cannot tell which area remains dirty, or do not want another round after finishing.
- **Do not infer:** Success proves only the *clearing* interaction; it does not prove a monetizable simulator or long-term retention.

2. Locked minimum scope (P0)

Area	P0 requirement	Explicitly excluded
World	One flat 48 x 48 stud yard, a start pad, one bounded cleanup plot (~24 x 24 studs)	Open world, biomes, quests, decoration pass

Area	P0 requirement	Explicitly excluded
Mess	60 dirt/debris patches initially arranged on an 8 x 8 logical grid (four empty), 3 simple visual variants	Physics-heavy trash, hundreds of loose parts, rarity
Core input	Point at a patch; hold/tap Clean; receive immediate localized feedback	Tool upgrades, multiple equipment types
Patch state	Dirty (2 valid hits) → Half-clean (1 hit remaining) → Clean (0)	Dirt textures that require GPU pixel painting
Progress	0–100% based on *total hits completed*; clear area visually	Coins, XP, inventory, shops, rebirth
Finish	One short clean-area celebration; “Try another round” or “Finish”	Daily rewards, leaderboards, forced replay
Replay	Reset same yard with one of 3 deterministic seeded layouts	Additional authored maps
Data	Server-authoritative round and patch counters in memory	DataStore, cross-session progress

First-pass tuning values (not final design rules): 60 patches x 2 accepted hits = 120 successful actions; server accepts at most 7 cleaning actions per second; reach <=12 studs; target first completion 45–90 seconds. Theoretical fastest action time is less than actual completion time because movement and aiming matter. Tune based on recorded sessions rather than forcing players to wait.

3. Player-facing interaction contract

Player entry: Spawn facing the dirty yard. A small label says **“Clean the yard: aim at the mess and hold Clean.”** A visible `0% CLEAN` counter appears; tool and nearest patch must be obvious.

Action: The player targets an unclean patch and activates the Clean action. A valid hit changes the exact patch (e.g., dark → pale → disappears). Soft particles and an unobtrusive sound confirm acceptance. Invalid targeting causes no progress. The player can move while cleaning.

Completion: On the last valid patch hit, the whole yard appears cleared and the HUD displays `100% CLEAN` with an optional brief before/after camera view. Show equally visible buttons **“Try another round”** and **“Finish.”** No performance pressure or timed event.

Replay: A new round seeds the same patch pool into another prevalidated arrangement. Reset visuals and counters atomically; do not spawn a second yard or accumulate effects.

Input by device

- **Desktop:** mouse look / reticle target; hold left mouse button or keyboard action to clean.
- **Mobile:** keep the crosshair centered, provide a dedicated large Clean touch button, allow normal camera movement with the other thumb. A tap mode is an acceptable accessibility alternative; measure which option participants use.

- **Gamepad:** optional stretch. The core interaction must not require fine pixel accuracy.
- **Feedback:** use patch disappearance and a numeric counter, not color alone.

4. Level and content handoff

Asset	Quantity / constraint	Acceptance
Yard base	1 simple rectangular surface	Dirty area visible from spawn; no navigation bottleneck
Debris prefabs	3 small variations reused 60 times	Clearly dirty against ground, no need for unique textures
Cleanable marker	60 server-tracked logical cells / models	Every cell stable ID and targetable collision/query proxy
Tool	1 simple broom / scrubber	Legible in first-person and third-person; simple feedback
HUD	1 percentage + prompt + completion panel	Readable on 16:9 desktop and portrait/narrow mobile viewport
Feedback	1 brief cleaning burst + 1 sound + finish effect	No heavy particle spam or camera shake
Replay layouts	3 saved seed patterns	No inaccessible patches / off-ground pieces

Spatial rule: The player should see an already-clean section building up next to a dirty section. This contrast is more important than impressive prop detail. Avoid piling patches into occluded corners. Use dark/bright value differences that work in outdoor lighting.

5. State, data and network contract

Round state machine

```
READY -> ACTIVE -> COMPLETE -> REPLAY_OFFERED -> RESETTING -> ACTIVE
```

Each `RoundState` holds `RoundId`, `LayoutSeed`, `PatchRemainingHits[PatchId]`, `TotalAcceptedHits`, `StartedAt`, and a config version. `TotalHitsTarget = 120` for the first test. Progress is derived as `TotalAcceptedHits / TotalHitsTarget`, **not** a number supplied by the client.

Server validation

A `CleanRequest(RoundId, PatchId)` requests one attempted clean. The server must verify, in this order:

1. Sender is live, alive, in the ACTIVE round, and is not in a reset transition.
2. The `RoundId` matches; `PatchId` belongs to the active yard and still has remaining hits.
3. Sender is within 12 studs of the patch and has a reasonably unobstructed interaction line to its query proxy.
4. Sender's action interval is at least `1 / 7 sec`; duplicate frames and spam do not skip states.

5. Decrement **one** hit, advance authoritative progress **once**, and publish the patch's new state.
6. When all patches reach zero, complete the round exactly once and block further progress.

The client cannot send its own percentage, force a finish, choose a new seed, or create valid patch IDs. Cosmetic local previews may appear immediately but must reconcile with server accepted/rejected messages.

Performance: Prefer a single raycast per active tool sample and small anchored/queryable patch proxies. Avoid 60 independent per-frame scripts, physics, or particle emitters. Cap active visual bursts and reuse effects. `CollectionService` tags (e.g., `RestorationPatch`) may identify spawned patch models. Keep the visual patches anchored and noncolliding so walking remains smooth.

Reset: Restore every patch's hit count and visuals, clear pending client feedback, rotate the `RoundId`, and reset the timer before accepting actions. Character reset mid-round returns the tester to READY (or a fresh round; choose one and keep it consistent).

6. Suggested small module organization

```
ReplicatedStorage/RestA/Config.lua
ReplicatedStorage/RestA/Remotes/CleanRequest
ReplicatedStorage/RestA/Remotes/PatchStateChanged
ReplicatedStorage/RestA/Remotes/RoundStateChanged
ServerScriptService/RestA/RoundService.lua
ServerScriptService/RestA/PatchService.lua
ServerScriptService/RestA/RequestGuards.lua
StarterPlayer/StarterPlayerScripts/RestAClient.lua
Workspace/RestA_Arena/Yard/PatchNodes/*
```

Names are illustrative. Use one bootstrap Script and a few modules; do not create generic services for hypothetical future products.

7. Production sequence and hard cuts

Step	Expected focus	Exit condition
A-01	Blockout yard, patch IDs and 1 cleaning action	One targeted patch changes state correctly
A-02	Server action validation, all 60 patches and progress	Spam / distant targets cannot skip progress
A-03	First-play HUD, particles/sound, completion/replay	Full round works end to end, resets correctly
A-04	Mobile control pass and 3 layout seeds	Can finish on phone without hidden instructions
A-05	Bug fixes and test logging	Acceptance suite passes, no added features

Cut order if late: three layout seeds -> decorative VFX -> extra tool animation.
Do not cut: visible local removal, complete transformation, replay choice, server authority, mobile action affordance.

8. Acceptance tests (A-T01 to A-T12)

ID	Test	PASS condition
A-T01	First entry	Spawn shows full dirty yard, obvious Clean action, `0%`
A-T02	First valid hit	Only targeted patch moves from dirty to half-clean; server hit tally +1
A-T03	Second hit	Same patch becomes clean; cannot be further cleaned
A-T04	Wrong/empty target	No progress and no phantom cleanup
A-T05	Too far / occluded	Forged request rejected; no progress
A-T06	Rapid-fire requests	Rate guard prevents accelerated progress; no double count
A-T07	Completion	Exactly 120 accepted valid hits resolve 100% and one completion event
A-T08	Replay	New RoundId restores visuals and counters; stale actions rejected
A-T09	Character reset / exit	No stuck held action or leaked event connections
A-T10	Touchscreen	A new player can move, aim and clean on a small touch layout
A-T11	Performance	60 patches and effects remain responsive in real/mobile emulation testing
A-T12	Second variation	Another valid seeded layout works without modifying code

9. Measurement and decision gate

Record common metrics in `05_Shared_Research_and_Playtest_Protocol_v1.0.md`: time to first valid hit, completion, replay choice, satisfaction, actual engineering hours, and authoring minutes for a second layout. Also record **time spent hunting the final 10%** and which patches were missed. The last dirty patch should not turn an otherwise relaxing game into tedious pixel hunting.

Proceed to deeper product research only if: most testers understand the clean action quickly, the ground visibly improves, a meaningful subset voluntarily replays, and authoring new layouts is genuinely inexpensive. **Stop or change approach if:** users simply sweep mindlessly without enjoyment, search time dominates, or the motion feels too imprecise.

10. Risks, fallbacks and AI guardrails

Risk	First fallback
Player cannot find last patches	Use subtle highlight on remaining patches at >=90% progress; record how often needed
Client requests spoof progress	Server owns cell list, distance checks, action rate, and completion
Cleaning feels like clicking checkboxes	Increase patch visual response before adding extrinsic rewards
Touch reticle too difficult	Increase query proxies; test tap-to-clean mode as an explicitly logged variant
60 patches create microstutter	Pool effects, reduce part count, remove needless collisions

Instructions to Claude / Codex implementing this document:

1. Do not modify *Thaw a Creature* files or shared Roblox places. Create a separate test place and inspect current content before editing.
2. Implement **P0 only**. Do not assume cash, upgrades, auto-cleaners, rarity, or inventories are approved.

3. Treat all target counts, ranges, durations and acceptance numbers as tunable experiment values; centralize them in Config.
4. Keep the server authoritative over patch state and round completion; do not trust client hit totals.
5. Do not report `PASS` from reading code alone: launch Studio and verify desktop/mobile behavior. If Studio is not accessible, state **NOT TESTED**.
6. Stop at the agreed implementation boundary. Submit actual observed time and defects before recommending expansions.

11. References

Technical API documentation:

- https://create.roblox.com/docs/workspace/raycasting
- https://create.roblox.com/docs/reference/engine/classes/CollectionService
- https://create.roblox.com/docs/reference/engine/classes/ContextActionService

Competitive inspirations are context only; none of their original creation times or complete production requirements are established by this prototype spec.

PROTOTYPE B — OBJECT ORGANIZATION

Roblox Minimum Functional Fantasy Specification | v1.0

Status: PROPOSED FOR OWNER REVIEW; NOT IMPLEMENTED / NOT PLAYTESTED
Date: 2026-11-06
Prototype code: REST-B
Primary fantasy: "I can turn a chaotic room into a neat, orderly one by putting objects where they belong."
Reference products (inspiration only): *Clean Everything*, *Clean The Library*
First-pass effort assumption: 3–5 focused developer-days (6 hours/day); playtest recruitment/time separate.
Shared authority: `05_Shared_Research_and_Playtest_Protocol_v1.0.md`. Separate from *Thaw a Creature*.

1. Decision this prototype must answer

Does sorting and returning scattered items provide an enjoyable decision and a strong enough visual transformation to motivate voluntary replay, without money, collectibles or game progression?

- **Hypothesis B-H1:** Recognizing the correct destination and snapping a messy object into place feels rewarding.
- **Hypothesis B-H2:** The content-authoring cost of new arrangements is acceptably low once pickup and slot systems are built.
- **Potential disconfirmation:** Moving each object feels like an unnecessary journey, destination rules are confusing, or the room reads as equally cluttered after completion.
- **Scope distinction:** This tests *order and classification*, not unconstrained interior decorating or a simulated housekeeping job.

2. Locked minimum scope (P0)

Area	P0 requirement	Excluded
World	One readable shop/stockroom approx. 36 x 32 studs	New stores, neighborhoods, furniture catalogs
Objects	Exactly 12 items: 3 each from 4 visually distinct categories	Inventory sizes, prices, rare items, hundreds of props
Destinations	Four shelves/bins; three fixed snap slots per shelf	Free placement, physics stacking, building tools

Area	P0 requirement	Excluded
Pickup	One object carried at a time; obvious pickup prompt	Throwing, object combat, carrying multiple
Placement	Correct-category destination accepts item and assigns an empty slot	Scoring penalties, selling, timed orders
Wrong placement	Soft rejection + clear category cue; item remains carried	Punishment, losing item, confusing reset
Progress	Accepted items placed / 12; visual room becomes ordered	Coins, XP, permanent collection
Completion	12/12 -> short clean-room reveal -> replay / finish	Extra levels, rewards, achievements
Replay	Same room, reshuffle the 12 starting positions among prevalidated spawn nodes	Procedurally generated shop interiors

First-pass timing assumption: 12 objects x ~5 seconds/object including pickup, pathing and placement = ~60 sec; aim 45–90 sec. Use layout adjustments if travel time dwarfs meaningful sorting.

3. Player-facing experience

Entry: The shop floor is visibly chaotic; colorful objects on the floor contrast with four tidy labeled shelf sections. Text: **"Put everything back on the right shelves."** A `0/12 PLACED` counter is always readable.

Identify: Every object has a clear silhouette and category symbol. Every destination has the same icon/text cue, so the task does not depend on color recognition alone.

Pick up: Near an item, use `Pick up` through a `ProximityPrompt`. The model appears attached to a simple carry anchor. The held object does not physically shove the player or block the camera.

Place: At the destination shelf, trigger `Place`. If the item's category matches, the server selects the lowest available authored snap slot and places the item, with a short click/animation, outline or sound. If not, play a gentle error cue; do not put it into the wrong slot.

Finish: With all 12 items correctly placed, the floor is visibly clear and shelves appear orderly. Show `12/12` and equal prominence `Try another round` / `Finish`.

Replay: Choose one of three shuffled starting arrangements with the same categories and slots. The system must not create an unreachable prop or put an item under another part.

Cross-platform input

Use server-handled ProximityPrompt events for pickup and place. Roblox automatically renders prompts for keyboard, gamepad, and touch, reducing custom input work. For narrow screens, prompts must not overlap excessively; show one closest actionable pickup, and one obvious placement prompt when near the target shelf. Avoid requiring drag-and-drop precision on mobile.

4. Art and level production brief

Asset group	First-pass count	Readability requirement
Single room	1	Starting mess visible from entrance; short uncluttered walk paths
Category shelves	4	Big labels and symbols; no ambiguous category overlap
Prop models	12, ideally 8–12 simple meshes assembled from reusable parts	Similar footprint; can all fit carried and in snap slots
Snap slots	12 authored `Attachment` /marker positions	No clipping / shelf overlap with correct models
Spawn nodes	12 per arrangement, at least 3 validated arrangements	No blocked doorways; items pickable
Carry anchor	1 per character (hand or torso attachment)	Readable in first and third person
HUD	Counter, held category hint, placement prompt, finish choice	Works on touch screen

Example categories (first-pass content, not thematic commitments): Books, Toys, Tools, and Food. Replace only if the new set preserves obvious distinctions and equal item counts. Do not use branded packaging or trademarks as shortcuts for understanding.

Layout rule: The room's before/after should read from the same camera spot. Floor mess becomes empty floor, and previously bare shelves become clearly organized. Avoid extra decorative clutter that conceals the transformation.

5. Data model and server-authoritative state

Round state

```
READY -> ACTIVE -> COMPLETE -> REPLAY_OFFERED -> RESETTING -> ACTIVE
```

Per round, store `RoundId`, `PropStates[PropId] = {CategoryId, State, HolderUserId?, PlacedSlotId?}`, `Slots[SlotId] = {CategoryId, OccupantPropId?}`, `PlayerHeldProp`, `AcceptedPlacements`, `LayoutSeed`, and `StartedAt`.

Prop lifecycle: `Scattered -> Held -> Placed`; `Held -> Scattered` on deliberate drop/character removal. An item may never be held by two players or placed in two slots. P0 can disable voluntary drop if its existence is not needed for completing the task; death/leave must still release or reset safely.

Authority for pickup and placement

Pickup (`ProximityPrompt.Triggered` on prop): Server validates ACTIVE round; original object belongs to this round; is `Scattered`; no item currently held; player's character is valid, alive, and <=10 studs from the object. Atomically mark `Held`, assign holder, disable old prompt, attach a non-colliding visual proxy with a server-authorized weld/attachment. If attachment fails, roll back ownership.

Place (`ProximityPrompt.Triggered` on shelf): Server validates ACTIVE round, player holds an actual item in this round, shelf is <=10 studs and correct category, at least one open snap slot exists. Reserve slot and mark prop `Placed` exactly once. Remove carry weld, anchor/snap the item to the authored slot on the server, increment `AcceptedPlacements`, and replicate state. The client may play cosmetic effects but cannot supply transform CFrames or decide valid slots.

Wrong shelf: Server rejects; sends clear UI feedback; **does not** consume held object or move it. Never allow negative inventory counts or ghost copies.

Server-only reset: Delete old carry constraints, remove old held visuals, reset all 12 props to validated spawn nodes, clear slot occupancy, create a new RoundId and counter, then reenable pickup. Late `Triggered` events from old round must not alter the new state.

Scope simplification

P0 uses anchored/non-colliding placed props and deliberately simple carry visuals. A full physics placement system is **out of scope**; adding it would confound whether players enjoy organization with whether physics feels stable.

6. Suggested file structure

```
ReplicatedStorage/RestB/Config.lua
ReplicatedStorage/RestB/Remotes/RoomStateChanged
ReplicatedStorage/RestB/Remotes/InteractionRejected
ServerScriptService/RestB/RoundService.lua
ServerScriptService/RestB/PropStateService.lua
ServerScriptService/RestB/PlacementService.lua
StarterPlayer/StarterPlayerScripts/RestBClient.lua
Workspace/RestB_Arena/SpawnNodes/*
Workspace/RestB_Arena/Shelves/*
Workspace/RestB_Arena/Props/*
```

Use the `ProximityPrompt`'s `Triggered(player)` on the server rather than trusting a client-sent `PropId` from arbitrary UI. Still validate distance, state, category, and round. This is a small prototype: avoid inventing an expansive inventory framework.

7. Production sequence and cut plan

Step	Focus	Exit test
B-01	Graybox room, shelves, prop IDs, 12 slots	Before/after composition already legible
B-02	Pickup one prop / simple server carry	Can't hold two, pickup from distance, or duplicate
B-03	Correct/wrong placement + fixed slots	Atomic placement and clean rejection
B-04	Full 12-prop content, progress and finish	One complete first round end-to-end
B-05	Mobile prompts, replay shuffle, logs, QA	All tests pass including reset and narrow screens

Cut first: three rearrangement seeds -> decorative labels / extra animation -> unique mesh variety. **Never cut:** category clarity, a proper correct-placement decision, visual room transformation, reliable carry cleanup, server state.

8. Acceptance suite (B-T01 to B-T13)

ID	Scenario	PASS condition
B-T01	First view	Mess and correct shelves recognizable without verbal tutorial
B-T02	Pickup	Exactly one available item attaches; original floor item isn't duplicated
B-T03	Wrong shelf	Rejected with clear feedback, still carrying the same item
B-T04	Correct shelf	Exact item snaps to first free correct slot, count +1 once
B-T05	Full shelf	No overwrite / cloning / loss; legitimate layout provides enough slots
B-T06	Second pickup while carrying	Rejected, item state unchanged
B-T07	Too far / invalid round	Trigger/replay exploit cannot move items or increase progress
B-T08	Simultaneous requests	One authoritative transition per object, not duplicate placements
B-T09	Character reset / leave	Carry constraints and state cleaned, no hovering stuck model
B-T10	12/12 finish	Single completion event, ordered room visibly better
B-T11	Replay	All items, slots, prompts and counter reset under new round
B-T12	Touchscreen	No prompt overlap prevents completion; no drag gesture requirement
B-T13	Content variation	Alternate starting arrangement requires no new code

9. Instrumentation and research decision

In addition to shared metrics, log **wrong-shelf attempts**, **average travel time per placement**, **time carrying an item without approaching any destination**, and **number of players who visually scan the room before choosing objects**. These reveal whether organization creates satisfying decisions or simply time-consuming trips.

Advance only if: sorting choices are understandable, the room looks unmistakably restored, replay is voluntary for a meaningful fraction of testers, and reorganizing the same content remains enjoyable. **Reconsider if:** confusion or traversal dominates, props are tedious to pick up, or making additional rooms requires more art than a small team can sustain.

10. Risks and AI guardrails

Risk	Mitigation
Correct location unclear	Match category icon/text on item and destination; verify with newcomers
Carry physics unreliable	No free physics placement; use server-owned attachment and fixed snap slots

Risk	Mitigation
Shelves block prompts	Test from actual camera distances; enlarge prompt target volume without enlarging prop
Wrong shelf feels punitive	Keep item held, gentle correction, no timer loss
Replay has no novelty	Change seeded scatter layout first; **do not** jump to inventory or coins
Content authoring slows	Measure second layout creation time; simple category props over bespoke asset catalogs

Instructions to AI implementers:

1. Use a separate Roblox test place. Do not touch *Thaw a Creature*, shared progression, or prior projects.
2. Do not implement drag-and-drop UI, inventories, money, object selling, multi-room levels, rebirth, or animation packages as unapproved embellishments.
3. Server owns which prop is carried, shelf category, occupied slot, and placement count. Never trust client CFrames.
4. Centralize first-pass counts and distances. Preserve full gameplay flow on narrow mobile screens.
5. A code review does not constitute Roblox Studio PASS. Explicitly report what was tested, which device, and remaining defects.
6. Obtain an explicit design decision before changing the fantasy from “put items in the right place” into decorating or construction.

11. Official API references

- https://create.roblox.com/docs/reference/engine/classes/ProximityPrompt
- https://create.roblox.com/docs/ui/proximity-prompts
- https://create.roblox.com/docs/reference/engine/classes/CollectionService

PROTOTYPE C — RESTORATION WITH A REVEAL

Roblox Minimum Functional Fantasy Specification | v1.0

Status: PROPOSED FOR OWNER REVIEW; NOT IMPLEMENTED / NOT PLAYTESTED
Date: 2026-11-06
Prototype code: REST-C
Primary fantasy: "Every section I restore lets me discover more of a hidden, surprising result."
Reference products (inspiration only): *Clean a Painting* and adjacent find-the-hidden-object experiences
First-pass effort assumption: 3–5 focused developer-days (6 hours/day).
Shared authority: `05_Shared_Research_and_Playtest_Protocol_v1.0.md`. Separate from *Thaw a Creature*.

1. Research decision

Does progressive uncovering of an unknown result make a simple cleaning action intrinsically replayable, even with no inventory, rarity, cash or permanent collection?

- **Hypothesis C-H1:** Players experience noticeable curiosity during restoration and voluntarily start another reveal.
- **Hypothesis C-H2:** New reveal outcomes are less expensive to produce than new 3D locations or mechanisms.
- **Falsification:** Players can guess every result immediately, clean only to finish rather than enjoying the act, or lose interest after seeing all three outcomes.
- **Important confound:** This prototype contains anticipation absent from A. To isolate its benefit later, optionally A/B test **known vs hidden result using the same restoration interaction**, but do not build that second condition without approval.

2. P0 scope lock

Component	Minimum requirement	Explicit exclusion
World	One small neutral gallery/workstation facing a large panel	Home base, multiple rooms, travel

Component	Minimum requirement	Explicit exclusion
Hidden content	Three original, rights-cleared illustrations at the same aspect ratio	Rarity tiers, branded images, UGC library, infinite generation
Cleaning surface	One 3:2 panel with **6 x 4 = 24 opaque cover tiles**	Real paint simulation, continuous pixel erase, fluid physics
Per tile	2 accepted actions: dirty -> partly clear -> revealed	Brush/pressure tiers, consumable soap
Interaction	Aim/tap tile; immediate localized uncovering	Purchasable cleaners, auto-cleaning
Result selection	Server selects one of 3 equally weighted outcomes at start	Client reroll, hidden monetary odds
Progress	Accepted cleaning steps / 48; 0–100%	Coins, XP, collection completion
Completion	Full picture visible, gentle reveal celebration, replay/finish	Save to gallery, trading, rarity announcements
Replay	Reselect outcome, reset 24 tiles, same panel	New buildings, quest lines

First-pass tuning assumptions: 24 tiles x 2 hits = 48 accepted actions; <=5 accepted actions/sec; <=12 studs to panel; 45–90 sec target including navigation and reading. A raw 48-action sequence can take much less; if measured rounds finish too quickly, change tile feel and movement gently, not add artificial waiting or a grind economy.

Content requirement: Prepare 3 original illustrations with visually different large silhouettes/compositions. Avoid trademarks and copyrighted characters. Don't describe the results with names visible before completion; the experience must preserve genuine curiosity without deceptive reward promises.

3. Moment-to-moment experience

Spawn: A framed panel is almost completely concealed by an opaque, obviously cleanable grime covering. Header: **“Restore the picture. What is underneath?”** A counter shows `0% REVEALED`. The three possible outcomes are **not** previewed.

Clean: The player points at a tile and holds/taps the cleaning input. On first valid hit, the cover fades to a still-obscuring partial state (e.g., cracked or slightly transparent). On second valid hit, the tile vanishes, exposing exactly that portion of the final illustration. Sound and particle feedback communicate an accepted action.

Curiosity: A revealed portion should be recognizable as part of a larger image, but the whole result is initially uncertain. Do not show the artwork in a thumbnail, nameplate, UI banner, folder label in-world, or immediate full preview. Note: a determined client can inspect replicated assets; the prototype has **no valuable hidden rewards**, so it does not claim cheat-proof secrecy.

Final reveal: At 48/48 accepted actions, all cover tiles are gone. Animate a short, non-flashing result presentation, followed by equally prominent **“Try another picture”** and **“Finish”** choices. Don't tell players that one outcome is rare or special.

Replay: Same workstation, fresh immutable `RoundId`, newly chosen result, reset tiles. With only three results, outcomes may repeat. Do not hide repeats or silently skew probabilities to manufacture anticipation. Record repeats in the test notes.

Mobile input

Prefer a simple clickable 3D panel with queryable thin cover tiles or a correctly placed `SurfaceGui`, using either a center reticle + Clean touch button or large directly tappable cover cells. Choose **one** pattern for P0, test it on narrow screens, and avoid requiring two simultaneous pinch/drag gestures. The panel should fill enough of the viewport for individual tiles to be confidently targeted.

4. Visual design and asset list

Asset	Quantity	Notes
Gallery / workbench room	1	Neutral background emphasizes before/after
Reveal frame/panel	1, 3:2 aspect ratio	One consistent size for all results
Original reveal artwork	3	Equal visual importance, rights-cleared, similar detail density
Cover tiles	24 (6 columns x 4 rows)	Opaque starting coverage; distinct partial-clean feedback
Cleaner/brush	1	Can be a simple effect with no elaborate arm animation
HUD	1 progress number + result/replay UI	Large type and touch targets
Sound/VFX	Brief tile clean, result finish	Low intensity, avoid bright flashing

Visual rule: Progress must be *local* and visually attributable. A random percentage increase without visible uncovered artwork fails the core fantasy. Avoid transparent gaps between untouched tiles that reveal the full image early. Test how much of the image becomes identifiable after 25%, 50%, and 75% cleanup.

5. Technical and authority contract

Data

`RoundState = { RoundId, State, ResultId, TileRemainingHits[1..24], AcceptedHits, StartedAt, ConfigVersion }`. The server chooses `ResultId` once at round start from a config list and does not reroll after the first action. `Progress01 = AcceptedHits / 48`.

Request

`RevealTileRequest(RoundId, TileId)` is a **request**, not proof of interaction. Server validates:

1. Sender's active round and exact RoundId; reject old or completed rounds.

2. TileId is within `[1,24]`, belongs to the player's active panel, and has remaining hits.
3. Character is alive and near panel (`<=12 studs`), facing the target surface, and the target is unobstructed from the player's viewpoint or approved proxy.
4. Action rate <=5 accepted hits/sec. A replay packet cannot finish an already complete tile.
5. Decrement tile state **once**, increment accepted hits once, replicate new tile status.
6. At 48 accepted hits, clear all tile presentation, mark COMPLETE once and present result.

Client presentation: Cover tiles can be separate lightweight parts, each initially opaque, with appearance determined by their replicated state. The underlying illustration can be placed via a `SurfaceGui` with an image asset. A screen-space canvas or a `SurfaceGui` is acceptable, but do not depend on a custom shader or image-editing engine. Use local animation only as cosmetic anticipation of server-accepted state; authoritative percent stays server-driven.

Visual secrecy caveat: If the result image asset is replicated to the client at start, technically sophisticated players can discover it before uncovering. For this **noncompetitive, non-monetized experiment** that is acceptable. Do not reuse this design for paid blind-box drops without a fresh security, policy and odds-disclosure review.

Reset: Server issues fresh RoundId, clears tiles and result UI, and selects one new preauthored ResultId. Late tile requests from earlier round are discarded. No DataStore; result ownership does not exist.

6. Suggested implementation layout

```
ReplicatedStorage/RestC/Config.lua
ReplicatedStorage/RestC/Remotes/RevealTileRequest
ReplicatedStorage/RestC/Remotes/TileStateChanged
ReplicatedStorage/RestC/Remotes/RoundStateChanged
ServerScriptService/RestC/RoundService.lua
ServerScriptService/RestC/TileRevealService.lua
ServerScriptService/RestC/ActionGuards.lua
StarterPlayer/StarterPlayerScripts/RestCClient.lua
Workspace/RestC_Arena/RevealPanel/TileProxies/*
Workspace/RestC_Arena/RevealPanel/PictureSurface
```

No custom EditableImage is required for P0. The 24-tile approach intentionally tests the experience without buying the engineering complexity of true smooth strokes.

7. Time-boxed build sequence

Step	Focus	Exit criterion
C-01	Gallery, panel, 24 cover nodes, 1 original picture	At least one tile cleanly reveals local picture portion
C-02	Server tile states and input validation	No invalid tile, distance, spam, or duplicate progress
C-03	3 outcome images, random server selection, finish	End-to-end reveal and accurate result selection
C-04	Mobile target sizes, feedback and replay reset	Second round works; clear difference between result/cover
C-05	Instrumentation, 25/50/75% readability check and QA	Acceptance suite passes without adding systems

If late, cut: decorative room geometry -> unnecessary per-tile particles -> additional result art beyond the required 3 only with owner approval. **Do not cut:** partial uncovering, three possible outcomes, real 100% transformation, replay, server authority, mobile controls.

8. QA acceptance suite (C-T01 to C-T14)

ID	Test	PASS condition
C-T01	Enter	Panel is genuinely obscured and progress starts 0%
C-T02	First cleaning action	Only targeted cover tile partially clears
C-T03	Second valid hit	Tile completely uncovers the matching artwork portion
C-T04	Different tile	Other tiles remain obscured and independent
C-T05	Invalid ID, distance, occlusion	No progress; server rejects request
C-T06	Burst spam / repeated cleared tile	Cannot double count or skip required hits
C-T07	Partial image clarity	At 25%, 50%, 75% progress, only cleaned regions reveal the result
C-T08	Outcome selection	All 3 authored images can be selected across seeded QA rounds
C-T09	No reroll	ResultId remains stable throughout a round and on invalid actions
C-T10	Completion	Exact 48 accepted hits -> one final reveal event, 100%
C-T11	Replay	New round ID, fully obscured surface, no old tile state
C-T12	Stale request	Pre-replay tile message cannot mutate fresh round
C-T13	Mobile play	Every tile targetable; no accidental camera/GUI dead zone
C-T14	Rights and presentation	Only original licensed art; no deceptive rare-reward claims

9. Measurement and next decision

In addition to shared metrics, record: **fraction of players who guess the hidden result before 25/50/75%**, first moment of visible excitement, whether participants clean beyond the moment they recognize the image, and whether outcome repetition harms the desire to replay.

To isolate the value of curiosity after P0, an optional controlled variant can display the identity of the result before cleaning. Keep the same tile interaction, same visuals and same participants protocol while rotating order. That tests *revelation versus pure restoration* more directly than comparing C to A alone.

Proceed only if: cleaning itself remains tolerable; uncertainty adds observed engagement; producing three original outcomes was manageable; and testers voluntarily start a second result. **Stop or revise if:** people recognize the picture too early, dislike the tile-by-tile approximation, or only want a collectible reward after the interaction.

10. Risks and implementer guardrails

Risk	Mitigation
Flat image reveal feels like a click puzzle	Smooth tile transition and add subtle cleaning motion; no currency shortcut
Surprise disappears after first play	Three genuinely distinct images; log repeat-result reaction
Clients read replicated results early	Accept for no-value P0; do not imply anti-cheat secrecy
One screen size makes tiles too small	Scale panel and interaction proxies; phone QA is a hard gate
Tile and artwork alignment wrong	Lock 3:2 aspect and row/column mapping; test edges and corners
Assets legally questionable	Use original works or rights-cleared images only

AI guardrails: Build only 24 fixed tiles / 3 equal-probability outcomes. Do not invent rare odds, collectible economies, image generation, secret trading, result storage, extra buildings, or a full painting editor. Keep server authoritative for tile acceptance. Explicitly mark any Roblox Studio or mobile behavior **NOT TESTED** until observed. Do not modify *Thaw a Creature*. Treat every numeric value as an experiment config, not an inferred market fact.

11. Technical references

- https://create.roblox.com/docs/workspace/raycasting
- https://create.roblox.com/docs/reference/engine/classes/ProximityPrompt
- https://create.roblox.com/docs/reference/engine/classes/CollectionService
- https://create.roblox.com/docs/reference/engine/classes/EditableImage (alternative only; not P0)

PROTOTYPE D — PRECISION WASHING

Roblox Minimum Functional Fantasy & Technology Spike | v1.0

Status: PROPOSED FOR OWNER REVIEW; NOT IMPLEMENTED / NOT PLAYTESTED
Date: 2026-11-06
Prototype code: REST-D
Primary fantasy: "Wherever I aim the washer, I can remove grime and reveal a clean surface."
Reference products (inspiration only): *Pressure Wash Simulator*, *Wash the Car*
First-pass effort assumption: 6–10+ focused developer-days (6 hours/day), **conditional on a 1–2 day technical spike**. Duration is not verified competitor production time.
Shared authority: `05_Shared_Research_and_Playtest_Protocol_v1.0.md`. Separate from *Thaw a Creature*.

1. Research decision — more fidelity, more risk

Does precise aiming and progressive surface cleaning deliver enough additional tactile satisfaction to justify its substantially greater engineering and asset setup cost compared with discrete clearing and reveal prototypes?

- **Hypothesis D-H1:** Aimed cleaning feels responsive and consistent enough that players enjoy the stroke itself, not just the finished state.
- **Hypothesis D-H2:** Once an object uses a cleanable surface scheme, new similar objects can reuse the technology at an acceptable content-authoring cost.
- **Disconfirmation:** The spray feels inconsistent, patch boundaries look artificial, aiming on mobile is frustrating, or new 3D objects require major per-object engineering.
- **Do not claim:** A tiled mock-up proves the feasibility of continuous paint-like surface removal over arbitrary curved meshes. The P0 prototype is deliberately an **approximation**.

2. Mandatory early technology gate (D-G0)

Time-box: 1–2 developer-days, included in the 6–10+ total estimate. Before implementing a full round:

1. Make one boxy vehicle side panel with a **6 x 4 local dirt-patch grid** (24 patches).
2. Raycast a washer stream at the panel; hitting the targeted patch changes it locally in two stages.
3. Test aiming with desktop mouse and a narrow mobile viewport.
4. Check whether cleaning reliably occurs at the place indicated by the spray and whether patch transitions look tolerable.
5. Measure actual frame time, bad targeting, client-server correction, and how long adding a second cleanable side panel takes.

Continue only if: The tester can deliberately choose which patch to clean, aim corresponds to the observed effect, and mobile use is workable without special instruction. If this is not achieved within the time box, **stop D** rather than building a car wash economy to compensate. A different technology approach requires a separate decision and revised budget.

3. P0 scope after the technical gate

Component	Minimum functionality	Exclusions
Vehicle	One stationary, low-poly, flat-panel van / boxy car	Driving, multiple vehicles, curved sports cars
Washable faces	Three rectangular cleanable faces (side, front, rear)	Arbitrary curved meshes; full texture UV painting
Dirt patches	72 total anchored surface patches: 24 per face	High-resolution per-pixel grime
Patch state	Two accepted spray passes per patch	Dirt thickness variations / chemical systems
Tool	One washer with directed beam + simple mist	Multiple nozzles, tanks, pressure upgrades
Input	Hold spray, aim at specific visible dirty patch	Gesture-only precision brush editor
Progress	144 accepted patch hits target 100%	Cash, tips, rating, automation
Completion	Clean van visibly transformed; replay/finish choices	Car collections, garages, sales
Replay	Reset dirt on identical van	New vehicle progression, upgrades
Server	Exact patch hit acceptance and progress authoritative	Client-controlled dirt percentage

First-pass tuning: three 6x4 grids; two hits/patch; action acceptance <=6.5 hits/sec; target <=13 studs; approximate 45–90 sec first round. Measure actual first-round time; do not assume 144 actions guarantees the target duration.

Scope honesty: This tests *spatial targeting on multiple 3D faces* with tiled dirt. It will not produce the smooth irregular streaks of a polished PC PowerWash-style simulation. Those require separate art/UV/rendering R&D and are not authorized by this P0 document.

4. Player-facing experience

Entry: One very dirty, stationary boxy van sits near the spawn. The clean/dirty contrast is immediately visible. A simple HUD says **"Wash the van — aim and hold Spray"**, with percentage progress.

Spray: The player aims the tool and activates Spray. A thin stream/tracer indicates direction; the targeted dirt patch becomes partially cleaned and then disappears after the required accepted samples. A short wet impact effect and low-intensity sound follow valid accepted hits. Hitting a clean patch, sky, ground or wall makes no progress.

Traversal: Player walks around the van to reach each face, rather than washing the whole vehicle from one angle. Keep enough space around the three exposed faces and avoid a roof panel that requires awkward jumping or camera clipping.

Finish: All targeted faces read clean. Display `100% CLEAN`, a short before/after camera framing, and neutral `Try another round` / `Finish` choices.

Mobile: A large `Spray` button is separate from camera movement. Target proxies are slightly more forgiving than their visual geometry, but cleaning still occurs where the user aims. If a phone player cannot target the top/edge patches, the test fails rather than silently using an area-wide cleaning mechanic.

5. Art, layout and technical asset brief

Item	Qty	Notes
Blockout van	1	Flat cleanable faces, readable silhouette
Washable face proxies	3	Side/front/rear; consistent local grid basis
Dirt patches	72	Visually flush to panels; queryable targets, no physical collision
Washer	1	Reticle, short stream, hit marker; no fluid simulation
Ground/test bay	1	Enough room for all three faces, neutral lighting
HUD / completion	1	Progress and touch-friendly finish/replay
Spray feedback	1 shared lightweight visual	Pool and cap effects; no persistent water particles
Alternate dirt layout	1 seeded reset	No need for a second vehicle

Important rendering rule: Dirt patches should not visibly float, flicker against the vehicle, or create collision barriers. Set up stable offsets relative to their cleanable face to avoid z-fighting; test in Studio under different camera distances. Use one material scheme and a clear half-clean state, not hundreds of unique textures.

6. State and networking model

Round data

`RoundState = { RoundId, PatchRemainingHits[1..72], AcceptedHits, TotalTargetHits=144, State, StartedAt }`. `Progress01 = AcceptedHits / 144` server-side.

The server is the only authority that decides which patch was hit and whether it counts. The client sends `SprayRequest(RoundId, PatchId, optionalAimRay)` as a **hint**, not a result. To avoid trusting arbitrary client `PatchId`, the server independently validates the candidate by geometry and reach. For first-pass P0, the server can calculate a line from a known valid player/head/tool region to the expected patch target and check obstruction and facing, rather than attempting to replicate exact camera input perfectly.

Minimum server guard contract:

1. Check live, active player and correct RoundId; no spray during replay/reset.
2. Verify patch exists in the active vehicle and is not already clean.
3. Require <=13 studs between legitimate character origin and patch; target is in a reasonable forward-facing cone and not obscured by another world surface.
4. Rate limit accepted patch hits to <=6.5 per second (config first pass). Reject duplicates, forged target IDs and stale packets.
5. Apply exactly one dirt-state step, then replicate accepted patch visual state and total progress.
6. Complete once at exactly 144 successful state changes; stop accepting action requests before replay.

Aim validation limitation: A server cannot trivially know the exact local player's camera ray at the exact input time. A client-supplied ray may help the server reconcile targeting, but must be normalized, range-bounded and checked against the authoritative world. Do not claim mathematically identical client/server aiming or use untrusted client hit results as proof.

Do not stream a RemoteEvent per render frame. Send a bounded number of requested spray samples while the input is held (e.g., 5–7 requests/s) and lerp/trail cosmetics locally. The server's accepted-hit sampling governs progress. For a noncompetitive prototype, brief local visual anticipation is allowed if reconciliation is visible and not misleading.

7. Optional fidelity branch — not P0

Roblox `EditableImage` provides runtime image editing but has memory budgets and security/permission restrictions, including a documented maximum image resolution and asset ownership requirements. A future image-based cleaning system would require robust UV or panel-coordinate mapping, appropriate performance tests and a new asset pipeline. Roblox Creator Hub documents those constraints; its availability does not make a production-ready texture-washing solution automatic.

Option D1 (P0): 72-part tiled dirt approximation (approved for *testing if implemented*).
Option D2 (unapproved future R&D): Fine-grained image-mask clearing for rectangular surfaces.
Option D3 (unapproved future R&D): Arbitrary curved-object UV/grime simulation.

Do not silently replace D1 with D2 or D3; they change both scope and risk.

8. Small implementation architecture

```
ReplicatedStorage/RestD/Config.lua
ReplicatedStorage/RestD/Remotes/SprayRequest
ReplicatedStorage/RestD/Remotes/PatchStateChanged
ReplicatedStorage/RestD/Remotes/RoundStateChanged
ServerScriptService/RestD/RoundService.lua
ServerScriptService/RestD/SprayValidationService.lua
ServerScriptService/RestD/SurfaceStateService.lua
StarterPlayer/StarterPlayerScripts/RestDClient.lua
Workspace/RestD_Arena/Van/CleanableFaces/*
```

Keep each patch anchored, stable and identified by one server-defined `PatchId`. All counters derive from server state. Client modules own camera reticle, stream VFX and UI only.

9. Production sequence with explicit exit gates

Step	Work	PASS to continue
D-00	Technology spike: one 24-patch side face	Accurate and satisfying local aim desktop + mobile; no serious stutter
D-01	Three cleanable faces; raycast targeting	Every patch can be reached without awkward movement
D-02	Server action guards and 2-hit patch state	No false hit, double progress, or remote spam bypass
D-03	Spray cosmetics, completion and replay	0–100% credible visual transformation and fresh reset
D-04	Mobile refinement, frame-time check, logs	Touch aiming succeeds; performance reasonable on available test device
D-05	Full acceptance pass, actual effort accounting	Decide whether fidelity is worth expanding

Cut order: decorative steam/water VFX -> complex tool model -> before/after camera animation. Do **not cut:** precise localized targeting, stable visual response, mobile aim, two-state surface transitions, server authority, completion/replay.

10. Acceptance suite (D-T01 to D-T16)

ID	Test	PASS condition
D-T01	Technical spike	24-patch face passes desktop aim and mobile aim, within time box
D-T02	Shot targeting	Only target patch responds; nearby patches stay unchanged
D-T03	Partial clean	First accepted hit visibly changes only that patch
D-T04	Clean result	Second hit fully removes patch and exposes clean face
D-T05	Missed ray	No server progress for sky, ground, clean area, out-of-range area
D-T06	Side/front/rear	All 72 patch IDs accessible and correctly oriented
D-T07	Camera edge tests	No erratic selection near corners or roofline
D-T08	Distant/occluded request	Server rejects forged or unreachable patch IDs
D-T09	Spam test	Maximum accepted rate enforced, no free progress
D-T10	Concurrent/stale packets	Exact one patch state transition per accepted hit; old round ignored
D-T11	Input hold/release	Spray starts/stops reliably; no stuck firing after respawn
D-T12	Completion	144 valid hits -> exactly one 100% event
D-T13	Replay	Dirt and progress reset, new RoundId, no old effects
D-T14	Mobile	Every face can be washed without hidden controls or pinching
D-T15	Performance	Real device or emulated narrow viewport tested; no persistent memory growth
D-T16	Scope honesty	Video of P0 shows tiled fidelity; no claim of smooth continuous washing

11. Decision and measurement

Alongside the shared protocol, record:

- Proportion of spray attempts whose **visual hit location matches player intent** (observation + tester question).
- Number of missed/incorrect target patches and time needed for the final 10%.
- Whether a user reports pleasure in precise strokes, or only in the final before/after.
- Engineering hours spent on the surface mapping/aim problem *alone*.
- Minutes to author and verify a second cleanable flat panel.
- Client FPS, device model/class, remote rejection counts, and reset defects.

Proceed beyond P0 only if: the precise interaction feels clearly better than coarse cleaning to testers, mobile controls hold up, and actual cost is defensible. **Stop if:** players mostly respond to the final clean state, because prototype A or C may supply that payoff much more cheaply.

12. Risks and AI guardrails

Risk	First response
Appearance too obviously tiled	Record as failure of fidelity; do not silently add custom shaders

Risk	First response
Spray doesn't match aim	Correct query proxies and authority/preview sync before visual polish
EditableImage temptation	Not approved in P0; separate R&D spike and permission/performance audit
Fine aim fails on phone	Widen query targets slightly while preserving local response; retest
Surface edges flicker	Offset surface patches and stabilize per-face alignment
New objects require manual mapping	Log authoring minutes; compare honestly with A/B/C

Instructions to Claude / Codex:

1. Begin with **D-G0 technical spike only**; do not build the full van before its result is approved.
2. Do not substitute a generic one-click wash, entire-face deletion, or global progress counter for localized washing.
3. Do not add vehicle driving, inventories, equipment upgrades, currency, a garage economy, robots, or collectible cars.
4. Keep patch state and progress server-authoritative. Consider a client ray only a validated hint.
5. Do not promise PowerWash-like continuous dirt removal from tile-based P0. Explicitly describe fidelity limitations.
6. Do not touch *Thaw a Creature*. Preserve all unrelated Roblox places and teammate code.
7. Report each acceptance test as PASS, FAIL or NOT TESTED based on actual Studio/client observation; never claim work was run when it was only designed.

13. Technical references

- https://create.roblox.com/docs/workspace/raycasting
- https://create.roblox.com/docs/reference/engine/classes/ContextActionService
- https://create.roblox.com/docs/reference/engine/classes/EditableImage
- https://create.roblox.com/docs/reference/engine/classes/CollectionService